Shelfwise

A library circulation platform, shipped like a production service

Aanvi Solanki · Roll no. 24BCS10170 · DevOps capstone (Session 21)



A real domain with real rules, not a CRUD demo

1

Tiered lending

Students 14 days × 5 books, faculty 28 × 10, public 14 × 3; renewal limits per tier

2

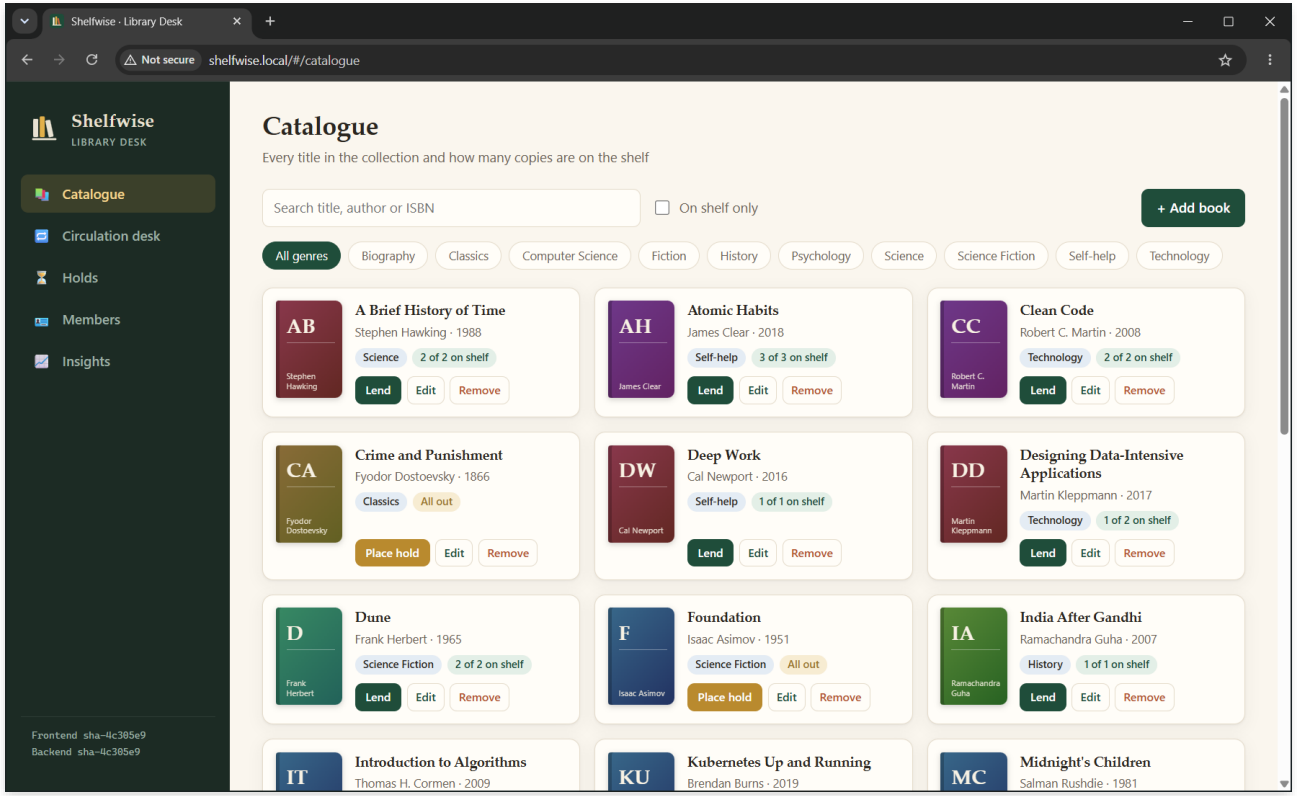
Fines

Grace period, per-day rate, per-loan cap; ₹150 owed blocks borrowing until settled

3

FIFO holds queue

A returned copy waits 3 days for the next member, then passes on



All rules are pure functions in backend/app/circulation/, tested with pinned dates

Every commit travels the same automated path

1

git push

main or a PR

2

Quality gates

ruff · 111 tests · build · helm · terraform · gitleaks

3

Trivy gate

0 HIGH / CRITICAL or nothing ships

4

GHCR

sha-<commit> tags, SBOM, provenance

5

Image Updater

sees the new tag in ~1 min

6

Argo CD

rolling update, zero downtime

No bot commits: Image Updater writes the tag to the Argo CD Application, not to Git, so every one of the 50+ commits is mine.

Proven in PR #1: the first commit added tests for a new rule; CI failed and skipped every image job; the second commit made it green.

Quality and security, measured

111

tests passing

98% coverage, isolated SQLite

0

HIGH / CRITICAL CVEs

Trivy on all three images

8

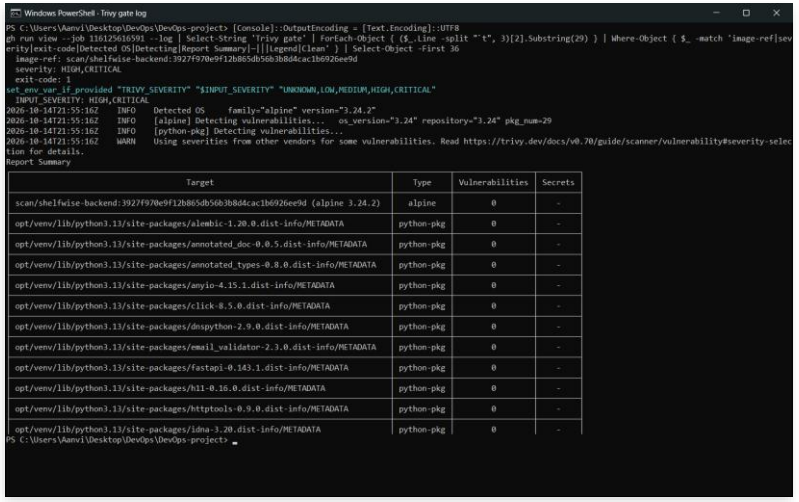
pipeline jobs

lint, test, build, helm, tf, secrets, images

10001

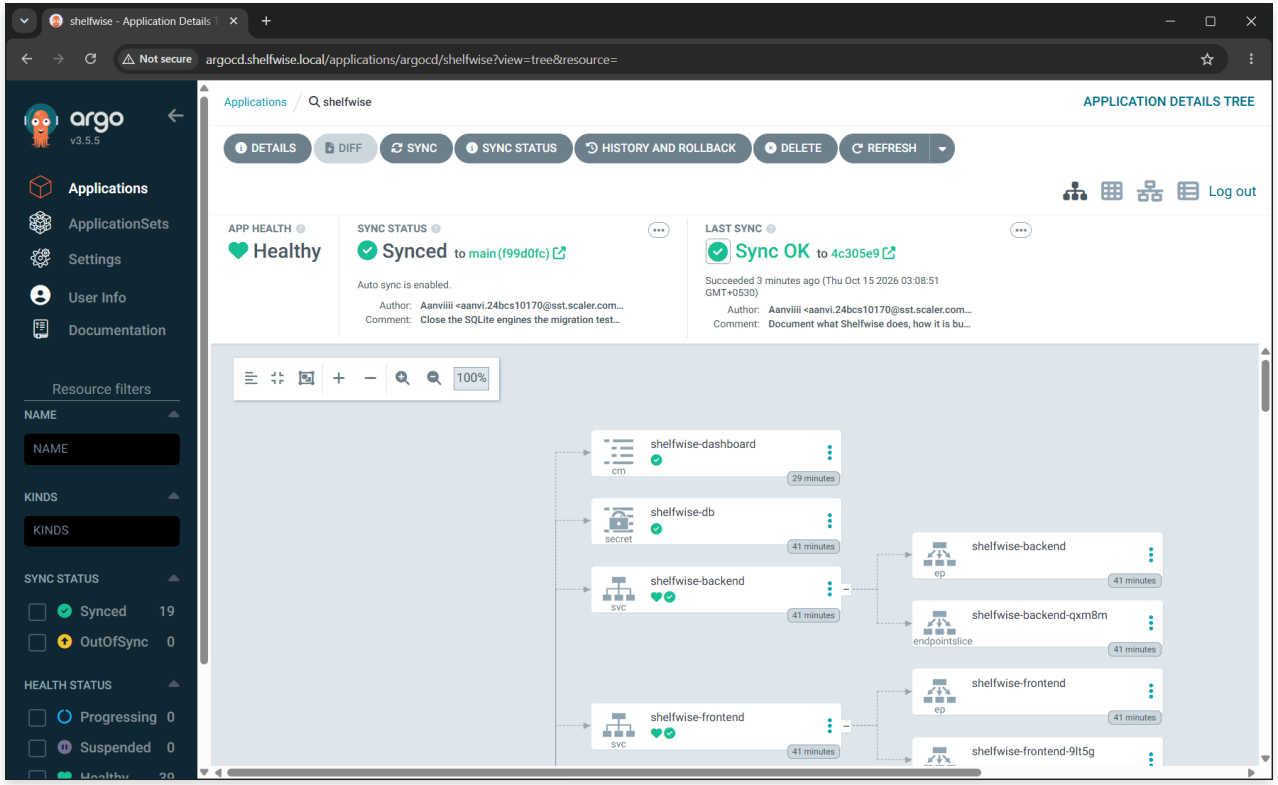
non-root UID

read-only root, all caps dropped



What Trivy checks: Alpine OS packages and every Python library in the image, plus secrets in the layers. Clean because runtime images are minimal (multi-stage, no pip, no Node). Re-scanned on every commit; Dependabot proposes updates weekly.

Kubernetes, packaged with Helm, delivered by Argo CD



Helm chart

Deployments, StatefulSet + PVC, ClusterIP Services, Ingress (/ and /api), PDB, NetworkPolicy, helm test

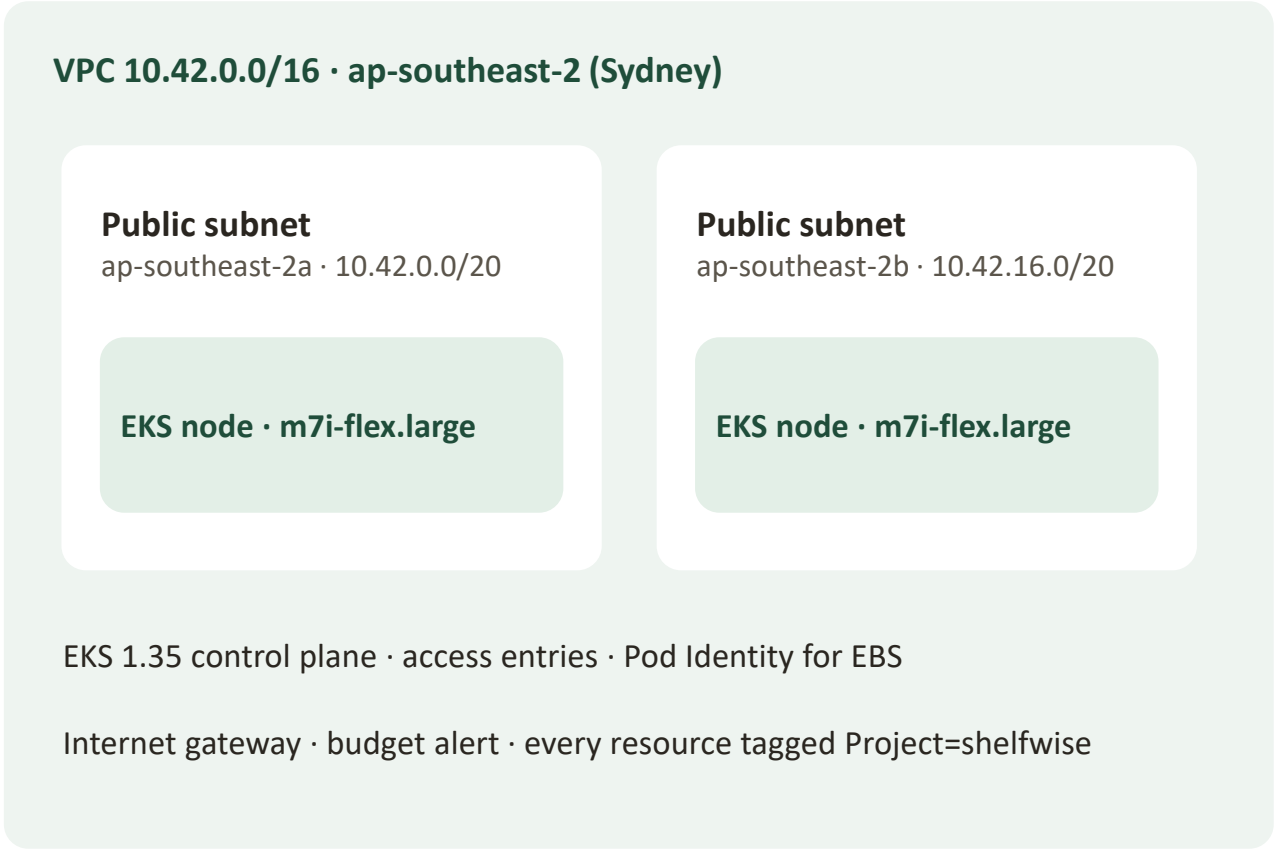
Autoscaling

HPA on CPU scaled the backend 2 → 6 pods under a k6 load test

Hardened

Restricted Pod Security on the namespace; probes split into startup, liveness and readiness

AWS with Terraform, priced for a student

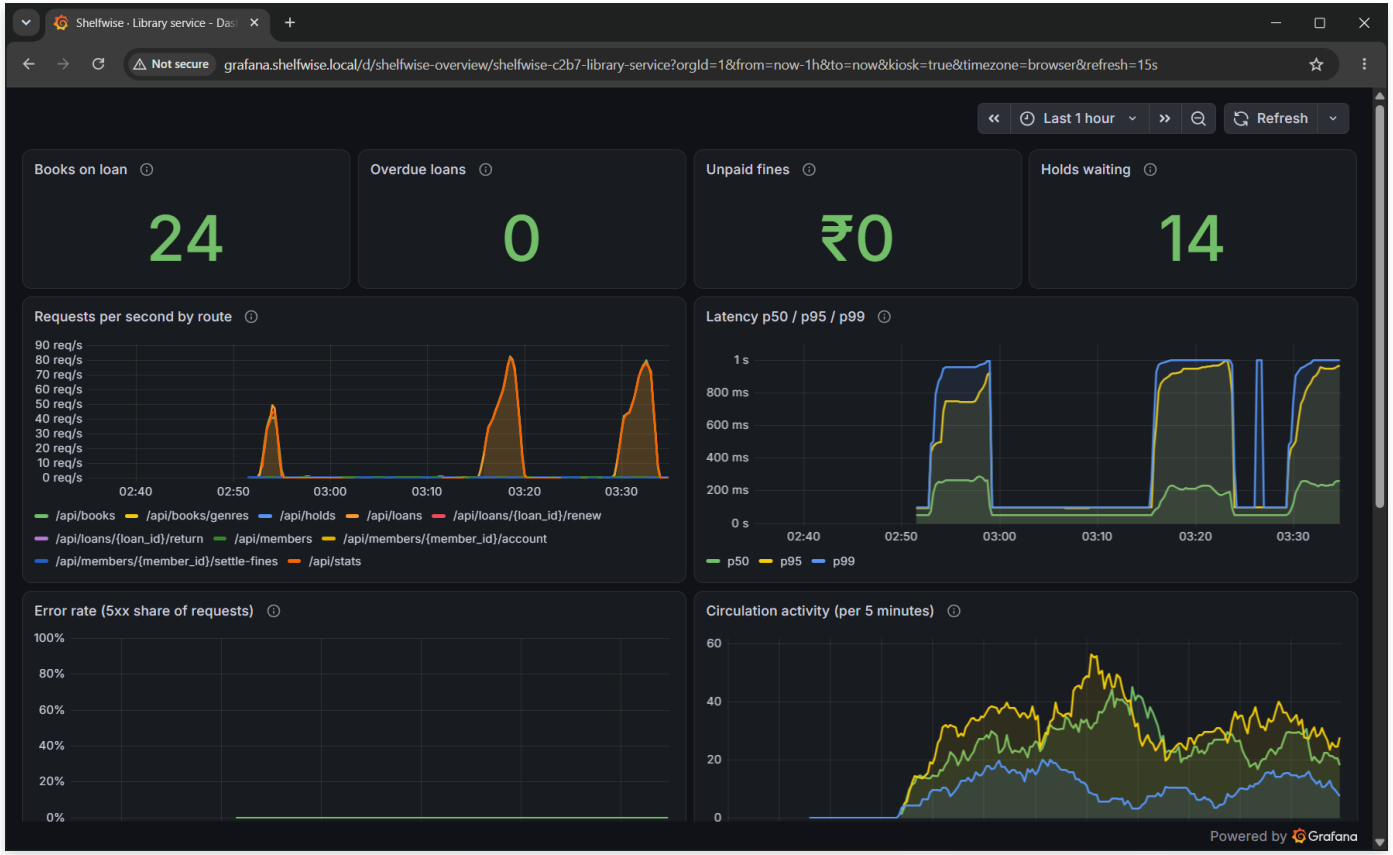


≈ $0.40

per hour, paid from AWS credits ($0.50 in total)

- No NAT gateway: saves ~$1.40/day
- Free plan: Sydney, On-Demand, Free Tier nodes
- Pinned to standard support (extended is 6×)
- Hand-written resources, no modules
- eks-down.ps1 removes LB + volume, then destroy

Observability: the library's health on one screen



HTTP metrics per route
Requests, p50/p95/p99 latency, 5xx ratio

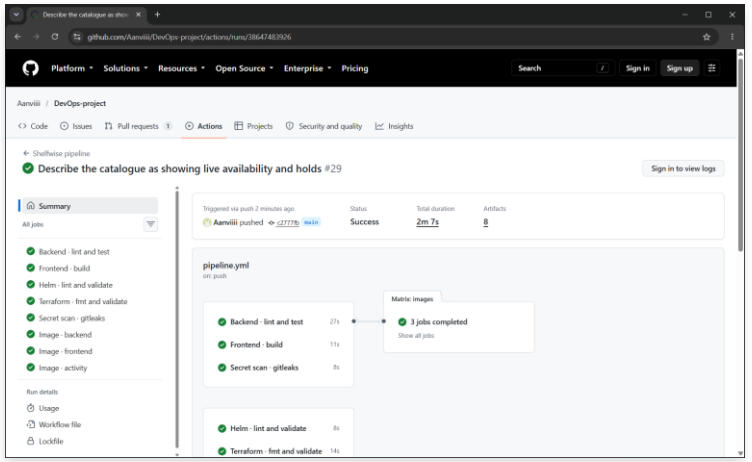
Library metrics from Postgres
Books on loan, overdue, unpaid fines, holds waiting, refusals by reason

Five alert rules
Backend down, error rate, slow requests, HPA at max, overdue backlog; two went pending under load

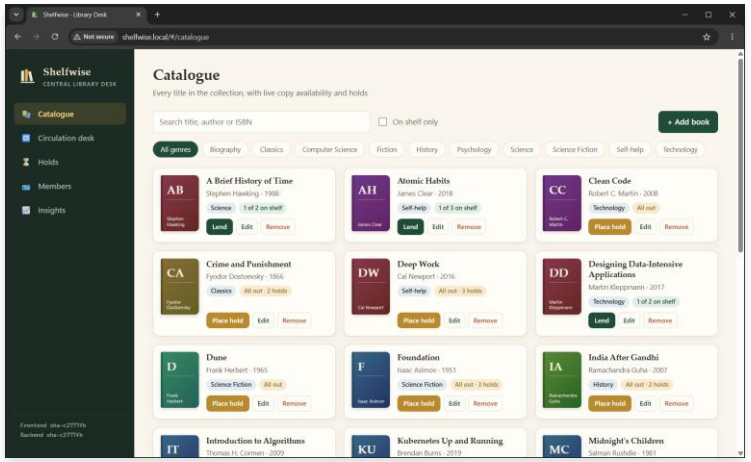
Live demo: one commit, no manual deploy



1 · Commit and push a UI change



2 · Pipeline: test, build, scan, publish



3 · Argo CD rolls out; the page updates

About five minutes from git push to the new version serving traffic. The sidebar and the X-Shelfwise-Version header name the running commit, so every pod traces back to its source.

What I would do next

- Move Postgres to Amazon RDS for backups and failover
- Replace ingress-nginx with the Gateway API
- Add staff login (OIDC) in front of the Ingress

Troubleshooting runbook: troubleshooting/RUNBOOK.md · Evidence: docs/SUBMISSION.md